



Pipelined datapath + control (slide 32, P&H): control in ID → WB/M/EX fields in ID/EX (9b), EX/MEM (5b), MEM/WB (2b).

```

define RI    0x00010000 // bits 29:16
define RD    0x00008000 // bits 15:11
define RDRN  0x00004000 // bits 10:6
define SHIFT 0x00002000 // bits 5:0
define OFFSET 0x00001000 // bits 15:0

int PC, IHW[1624], DMW[1624], RF[32], ALUControl;
bool ZF0;

int SE(int x){ return (short)x; } // 16-32 bits
int ALU(int x, int y, int op){
    case 0b0110: ZERO:=x<y?0; return x<y; // sub/B
    case 0b0101: return x+y; // ADD
    case 0b0100: return x-y; // sub/B
    case 0b0001: return x|y; // OR
    case 0b0011: return x&y; }
Load(MW, DMW); // decode of 1st instruction;
while(1){ int I = IHW[PC];
    switch(I & 0PCODE == 26){
        case R: ALUControl = (I & from); // ADD, SUB, AND, OR
            RF[I[IR] > 21] = ALU[RF[I[IR] > 21], RF[I[IR] > 16]];
            PC = PC + 4; break;
        case SW: ALUControl = 0b0010;
            DMW[RF[I[IR] > 21]] = ALU[SE(I[IR] > 21), SE(I[IR] > 16)];
            PC = PC + 4; break;
        case LW: ALUControl = 0b0010;
            DMW[RF[I[IR] > 21]] = DMW[RF[I[IR] > 21], SE(I[IR] > 16)];
            PC = PC + 4; break;
        case B: ALUControl = 0b0110;
            ALU[RF[I[IR] > 21]] = DMW[RF[I[IR] > 16], RF[I[IR] > 11]];
            if (ZERO == 0){ PC = (PC+4) & (SE(I[IR] > 21) & SE(I[IR] > 16)); }
            else PC = PC + 4; break;
        case J: ALUControl = 0b0010;
            RF[I[IR] > 16] = ALU[RF[I[IR] > 21], SE(I[IR] > 16)]; // homework
            PC = PC + 4; break;
        case PC = ((PC+4) & 0xF0000000) | ((I & 0x3FFFFFF) <<
2);

```

```

case 0: return SI[2]; // case 00000: return SI[2];
case 1: return SI[1]; // case 00001: return SI[1];
Load(MM, DM0); PC = address of first instruction;
while(1){ int i; i = MM[PC];
switch( i & 0FCE){ // case 0: R_OPCODE = 0 from funct; // ADD, SUB, AND, OR
RF[(IART)>=11] = ALU(RF[(IARS)>21], RF[(IART)>16]);
PC = PC + 4; break;
case SW: ALU_COMMENT = 000010;
DMW[ALU(RF[(IARS)>21], SE(I&0FSET))] = RF[(IART)>16];
PC = PC + 4; break;
case LW: ALU_COMMENT = 00010;
RF[(IART)>16] = DMW[ALU(RF[(IARS)>21], SE(I&0FSET))];
PC = PC + 4; break;
case B: ALU_COMMENT = 000110;
ALU(RF[(IARS)>21], RF[(IART)>16]);
if (ZERO == 1) PC = PC + 4 + (SE(I&0FSET)<<2);
else PC = PC + 4; break;
case ADU: ALU_COMMENT = 000110; // no hardware
RF[(IART)>16] = ALU(RF[(IARS)>21], SE(I&0FSET));
PC = PC + 4; break;
case J: PC = ((PC+4) & 0xF0000000) | ((I & 0x3FFFFFF) <<
2);

```

```

PC[16]>0x11 = ALU(RF((I&RS)>21), RF((I&RT)>16));
RF = PC + 4; break;
CASE SW: ALU(RF((I&RS)&0b010),
DMA[ALU(RF((I&RS)>21), SE(I&OFFSET))]) = RF(I&I&0x16);
PC = PC + 4; break;
CASE LW: ALU(RF((I&RS)&0b010),
RF((I&RT)>16)) = DMA[ALU(RF((I&RS)>21), SE(I&OFFSET))];
PC = PC + 4; break;
CASE RTYPE: ALU(RF((I&RS)&0b010),
ALU(RF((I&RS)>21), RF((I&RT)>16)));
if (ZERO == 1) PC = (PC+4) & (SE(I&OFFSET)<<2);
else PC = PC + 4; break;
CASE ADDI: ALU(RF((I&RS)&0b010), // immediate
RF((I&RT)>16)) = ALU(RF((I&RS)>21), SE(I&OFFSET));
PC = PC + 4; break;
CASE J: PC = ((PC+4) & 0xF0000000) | ((I & 0x3FFFFFF) <<
2);

```

```
case ADDI: ALUControl = 0b0010; // homework
RF[(I&RT)>>16] = ALU(RF[(I&RS)>>21], SE(I&OFFSET));
PC = PC + 4; break;
case J: PC = ((PC+4) & 0xF0000000) | ((I & 0x3FFFFFF) <<
2);
```

```
R14 Prof's C++ -- ulti-cycle (M1-d slide 15): one memory MM, ALU reused
int PC, MM[1024], RF[32], ALUControl; bool ZERO;
int IR, A, B, ALUOut, MDR; // non-architectural
Load(MM); PC = address of first instruction;
while(1){
    IR = MM[PC]; ALUControl = 0b0011; PC = ALU(PC, 4); //T0
    A = RF[(IR>>20)&31]; B = RF[(IR>>16)&31]; //T1
```

```

switch((IR & OPCODE) >> 26) { // T2
    case RTYPE: ALUControl = from func; // T6 (T2)
        ALUOut = ALU(A, B); // T7 (T2)
        RF[(IR&RD)>>11] = ALUOut; break;
    case LW: ALUControl = 0b0010; // T2
        ALUOut = ALU(A, SE(IR&OFFSET)); // T3
        MDR = MM[ALUOut]; // T4
        RF[(IR&RT)>>16] = MDR; break; // T3
    case SW: ALUControl = 0b0010; // T2
        ALUOut = ALU(A, SE(IR&OFFSET)); // T2
        MM[ALUOut] = MDR; // T2 (T2)

```

```

case B_TYPE: ALUControl = 00b018; // T0
ALUOp = ALU(PC, SE(IR[6:0]))<<2; // T1
ALUControl = 00b018; ALUOp = 0; // T8
if (IR[10:9] < 3) PC = ALUControl; // T9
case ADDI: ALUControl = 00b018;
ALUOp = ALU(A, PC[31:0] + SImm); // T2
RF[IR[5]] <- ALUOp; // T3
case J: PC = (PC & 0xF0000000) | (((IR[6:3] << 1) << 12)); // T3
// cycles: lw, sw, 4, r, 4, add, 4, beq, 3, j 3

```

RIS Multicyle datapath in Verilog (Harris)

```

module datapath(input clk, reset, PCgen, Iord,
IRwrite, RegDst, MemtoReg, Rgwrite, ALUSrcA,
input [1:0] ALUSrcB, PCsrc, input [2:0] ALUctrl,
input [31:0] ReadData, input [31:0] Op,
input Zero, output Zero, input [31:0] Addr, WriteData);
wire [31:0] PC; PCHext, Instr, data, R02, A, Sra,
SraC, ALUSrcA, ALUSrcB, ALUOp, ALUOut, wire [4:0] Wd3;
flopmem # (32) pcreg(clk, reset, PCgen, PCsrc, PC);
mux2 # (32) adrmux(PC, ALUOut, Iord, Addr);
flopmem # (32) r1r1clk, reset, IRwrite, ReadData, Instr);

```

```

mux2 # (5) wamux (Instr[20:16], Instr[15:11],
           RegDst, WA3);
mux2 # (32) wdmux (ALUOut, Data, MemtoReg, WD3);
regfile rf (clk, RegWrite, Instr[25:21], Instr[20:16],
           WA3, WD3, RD1, RD2);
signext se (Instr[15:0], SignImm);
floprr # (32) areg (clk, reset, RD1, A);
floprr # (32) breg (clk, reset, RD2, WriteData);
mux2 # (32) srcamux (PC, A, ALUSrcA, SrcA);
mux2 # (32) srcbmux (WriteData, 32'd 4, SignImm,

```

```
alu alu(SrcA, SrcB, ALUCtrl, ALUResult, Zero);
flop # (32) aluoutreg(clk, reset, ALUResult, ALUOut);
mux3 # (32) pcmux(ALUResult, ALUOut,
    SrcC[31:20], Test[0:31], alu0, PCNext, PCNext);
```

```

PC[31:28],Instr[29:0],z,0000,PCSrc,PCNext);
assign Op=Instr[31:26]; assign Funct=Instr[5:0];
endmodule
// controller: PCEn = PCWrite | (Branch & Zero)
// flopper: always @(posedge clk, posedge reset)
// if(reset) q<=0; else if(en) q<=d; {flopr: no en}
// regfile: write @posedge if we3; rd = {ra1=0}?rf[ra]:0

```

Computable = an algorithm exists (is it prime?); **uncomputable/unknown**: is a number random? **P**? **NP**. **Halting problem**: assume $A(P,D)$ decides if P halts on D : build $B(X)$: loop forever if $A(X,X) = \text{"halt"}$ else halt; $B(B)$ contradicts $\Rightarrow A$ doesn't exist (self-referential). **General-purpose** = programmable (Turing model) via the **fetch-and-execute algorithm**: stored program, generalized datapath & FU (von Neumann): fetch instr from memory $\rightarrow$ decode (opcode) $\rightarrow$ execute $\rightarrow$ PC+4. **Special-purpose** (Min/Max) runs one algorithm, no opcode. **Half adder**: $S = A \oplus B$, Cin . **Full adder**: $S = A \oplus B \oplus Cin$, $cout = AB + AC + BC$. **4-bit block**: 474 of 65536s.

High-level construct	Digital construct	Type
Scalar/variable	register or wire	sequential if reg
Array	memory (write: decoder, read: mux)	sequential
Operators + - /	functional unit	combinational
If-else / switch / loop	control unit	comb./sequential
If(sel) a=10;	multiplexer	comb.

else a==5;		
if(sel) a=10;	decoder (enables)	comb.
else b=3;		
for(i=10;i>0;i--)	counter (load max, decrement, zero → stop); nested loops: 2 counters, stop = AND of zeros	sequential
a<b, a==b	n-bit comparator (A>B, A=B, A<B)	comb.
module d_latch(input d, en, rstn, output reg q);		

```
always @(en or rstn or d)
    if (!rstn) q <= 0; else if (en) q <= d;
endmodule

module dff(input clk, d, output reg q);
    always @(posedge clk) q <= d;
endmodule
```

Abstraction layers: Problem → Algorithm → Program/Language → Runtime (OS, VM) → ISA → Microarchitecture → Logic (CS F342 focus) → Devices → Electrons. ISA = the computer as seen by the compiler (MIPS, RISC-V, x86, x86_64). MIPS = Microprocessor without Interlocked Pipelined Stages. Architecture = computational structure w/rt. user; minimal resources, scalable. Two ideas: all computers compute the same things given time & memory; problems must be transformed from human language to voltages. Moore's law: transistors per chip double ~2005. Computer history manual: mechanical → electro-mechanical → electrical (vacuum tubes → transistors). FPGA: CLBs of 4-input LUTs. Homework: bubble-sort SPP. GP processor with ADD, SUB, LR, SW, ROP, 32 regs, RO = 0.